LM Stats Suite
Technical Documentation

Comprehensive user and technical manual for the Streamlit application, including page structure, numerical engines, equations, libraries, and implementation notes.

Source reviewed: current LM-Stats-Suite.zip provided in the conversation.
Primary entry command: streamlit run app.py

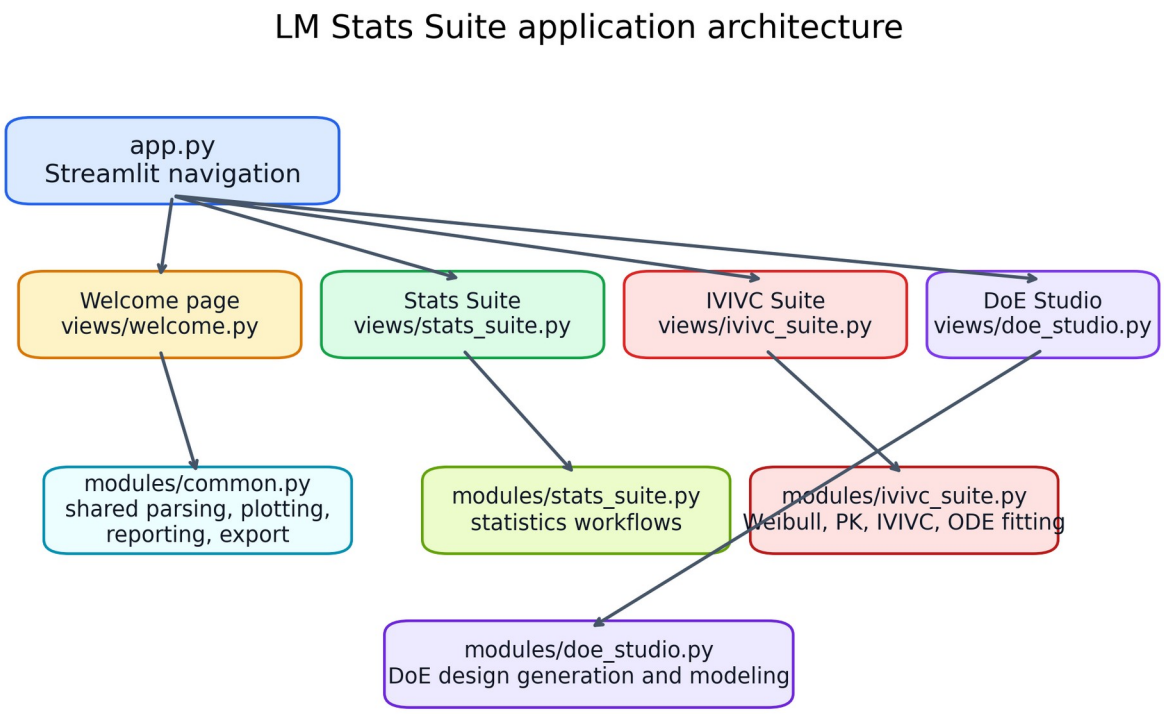

Figure 1. High-level application architecture and page-to-module relationships.

Contents

1. Application overview
2. Shared infrastructure and common utilities
3. Stats Suite
4. IVIVC Suite
5. DoE Studio
6. Reporting, export, and table-copy workflow
7. Python packages, key commands, and runtime APIs
8. Persistence, saved models, and session behavior
9. Appendix A — Module inventory
10. Appendix B — Quick operator checklist

The PDF version of this manual is bundled with the updated project zip. The Welcome page can also expose a lightweight Help index and a direct PDF download button.

1. Application overview

LM Stats Suite is a multipage Streamlit application for statistics, dissolution / IVIVC modeling, and design-of-experiments work. The root entry point is app.py, which calls Streamlit navigation and mounts four pages: Welcome, Stats Suite, IVIVC Suite, and DoE Studio. Each page script is intentionally thin and delegates the real computational work to a corresponding module under modules/.

The overall software pattern is: parse pasted tables → transform data into pandas/numpy structures → call numerical routines from SciPy / statsmodels / scikit-learn → render figures with matplotlib → export tables to Excel or PDF through openpyxl / reportlab.

File	Role	Observed size / scope
app.py	Application entry point; configures page navigation with st.navigation and st.Page.	16
modules/common.py	Shared parsing, plotting, statistical helpers, PDF/Excel export, and table rendering.	1134
modules/stats_suite.py	All statistics page workflows and plots.	1818
modules/ivivc_suite.py	Dissolution comparison, Weibull fitting, deconvolution, IVIVC fitting, and saved-model helpers.	3434
modules/doe_studio.py	Design generation and response-surface modeling workflows.	2609

The requirements.txt file lists the runtime package stack used by the app: streamlit, pandas, numpy, matplotlib, scipy, statsmodels, scikit-learn, openpyxl, reportlab.

2. Shared infrastructure and common utilities

The common module is the application backbone. It exposes page styling, table parsing, graph styling, export helpers, and low-level numerical utilities reused across all suites. The same module also keeps the visual language consistent: headers, metric cards, report-style tables, figure styling, and clipboard copy controls are all centralized here.

Important shared commands and APIs:

Command / API	Where used	Purpose
st.navigation([...]) / st.Page(...)	app.py	Registers the app pages and sidebar navigation entries.

inject_css()	all pages	Injects custom CSS for the visual theme, cards, and table styling.
render_display_settings()	Stats Suite, IVIVC Suite	Opens the sidebar display / export settings and allows per-plot styling.
parse_pasted_table() / parse_xy()	common, Stats, DoE	Converts pasted text into pandas DataFrames or numeric x–y arrays.
report_table()	all analytical pages	Renders a formatted HTML table in Streamlit and exposes a copy-table control.
export_results()	Stats, IVIVC, DoE	Builds Excel and PDF exports using openpyxl and reportlab.
apply_ax_style()	all plots	Applies border, legend, font, limit, and tick styling consistently.

Numerical helper functions in common.py include tolerance interval calculation, linear regression fit and prediction routines, residual and Q–Q plots, dissolution similarity support (f2, bootstrap, BCa interval), and drawing of confidence ellipses for PCA.

Export stack:

Excel output is built with openpyxl-compatible data packing through make_excel_bytes(). PDF output is created by make_pdf_report(), which uses reportlab SimpleDocTemplate, Paragraph, Table, TableStyle, Image, and PageBreak. This means the app produces publication-friendly reports without relying on the browser to print the page.

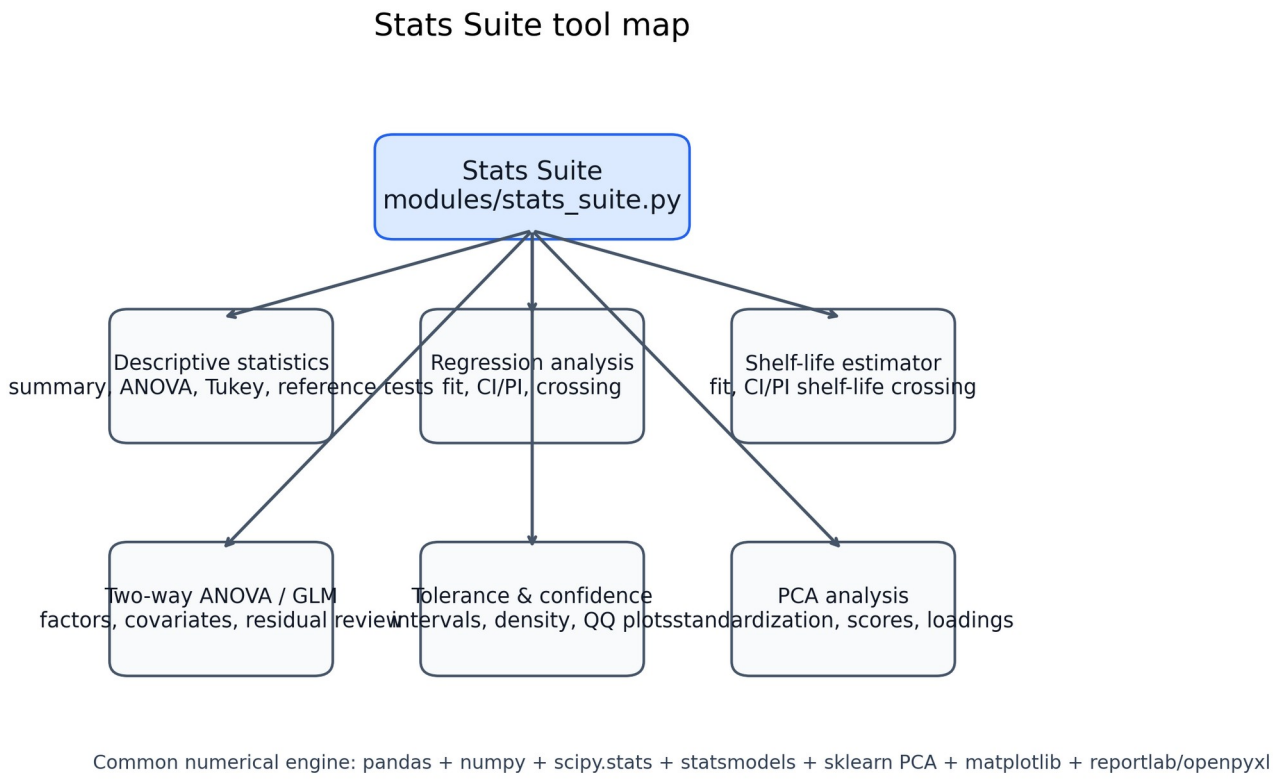

Figure 2. Summary map of the statistics workflows exposed through the Stats Suite page.

3. Stats Suite

Stats Suite is selected from the sidebar radio group defined in modules/stats_suite.py. The tool list currently includes Descriptive Statistics, Regression Analysis, Shelf Life Estimator, Two-Way ANOVA / GLM, and PCA Analysis. The page is designed for pasted tabular data; each tool validates numeric columns and then routes to a dedicated workflow.

3.1 Descriptive Statistics

Purpose: summarize one or more samples, compare selected samples against a reference, calculate mean confidence intervals and tolerance intervals, perform normality checks, and provide visual distribution summaries.

Main calculations in code:

Calculation	Implementation detail	Primary Python API
Descriptive summary	N, sum, mean, standard deviation, variance, quartiles, min/max, range.	numpy / pandas
Normality review	Anderson–Darling and Shapiro–Wilk p-values.	statsmodels.stats.diagnostic.normal_ad, scipy.stats.shapiro
Intervals	Mean confidence interval and normal-theory tolerance interval (two-sided or one-sided).	scipy.stats t / normal helpers + custom tolerance formula
Reference comparisons	Equal-variance checks, paired/unpaired comparisons, nonparametric alternatives.	SciPy tests + custom decision logic
All-sample comparison	One-way ANOVA + Tukey HSD plot when three or more groups are present.	scipy.stats / statsmodels.stats.multicomp.pairwise_tukeyhsd

The graphical summary combines box-plot style statistics with interval overlays and density shape information, giving a compact “one figure” diagnostic for lot-to-lot comparisons. When several groups are present, the suite also produces a Tukey HSD graphic to assess all pairwise differences, not just reference-vs-test comparisons.

3.2 Regression Analysis

Purpose: fit a linear model to an x–y dataset, generate ANOVA and coefficient tables, calculate confidence and/or prediction intervals, and optionally estimate where the fitted line or its interval band crosses a specification limit.

Core commands include the local routine regression_anova_and_coefficients_local(), the shared linear regression helpers from common.py, and advanced matplotlib plotting through plot_regression_advanced(). The regression workflow is appropriate for assay trending, calibration relationships, and simple time-vs-response models.

3.3 Shelf Life Estimator

Purpose: use the same linear fit machinery for stability trending, then estimate shelf life from the fitted mean line, the CI band, or the PI band crossing a lower or upper specification. The resulting shelf-life estimate is more conservative when based on confidence or prediction intervals than when based on the raw fitted line.

3.4 Two-Way ANOVA / GLM

Purpose: analyze two-factor experiments and generalized linear model layouts with factors and covariates. The tool creates formulas for statsmodels, fits an OLS model with smf.ols(...).fit(), then uses anova_lm(..., typ=2) for the ANOVA table. Residual and Q–Q plots are produced for model review.

3.5 PCA Analysis

Purpose: standardize selected numeric variables, compute principal components, and visualize score and loading plots. The implementation uses sklearn.decomposition.PCA with two components. Scores are computed from the standardized matrix $Z=(X-\text{mean})/\text{SD}$, while loadings are computed as $\text{components}^T \cdot \text{sqrt(eigenvalue)}$. Optional group ellipses use a covariance ellipse helper in common.py.

4. IVIVC Suite

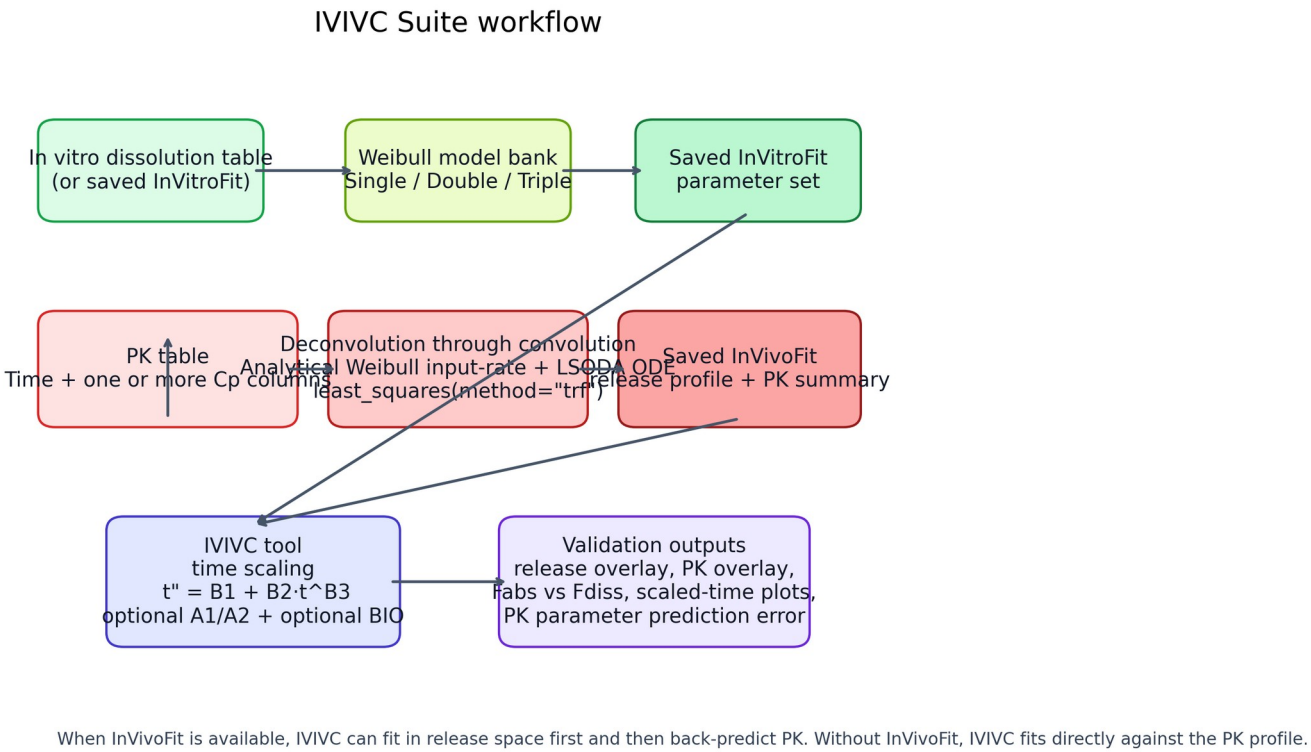

Figure 3. Major data flows inside the IVIVC Suite, from in vitro fitting to deconvolution and IVIVC time scaling.

The IVIVC Suite is the densest computational page in the application. It contains four tools: Dissolution Comparison (f_2), In Vitro Weibull Fit, Deconvolution through convolution, and IVIVC. The module imports scipy.optimize.least_squares for nonlinear fitting and scipy.integrate.solve_ivp for ODE simulation, then layers a set of pharmacokinetic and dissolution-specific helpers on top.

4.1 Fit objective and optimizer

All nonlinear fits in this module are built on scipy.optimize.least_squares(method="trf"). TRF stands for Trust Region Reflective. It is a bounded nonlinear least-squares algorithm; it is not classic Newton and not Levenberg–Marquardt. Where dynamic PK prediction is needed, the state equations are solved by solve_ivp(method="LSODA").

The weighted/log residual engine is shared across in vitro fitting, deconvolution, and IVIVC:

Option	Code behavior
Linear scale	Residual = sqrt(weight) × (predicted − observed).
Log(Y) scale	Observed and predicted values are log-transformed with a positive floor before residuals are computed.
Observed Y^-p	Weight base is the observed value; weight ∝ y^−p.
Predicted Y^-p	Weight base is the predicted value; weight ∝ ŷ^−p.
Normalization	Weights are normalized so their valid entries sum to the number of valid points, preserving practical scale.

This implementation mirrors a Phoenix/WinNonlin-style weighted least-squares workflow: the selected power changes how strongly low values are emphasized, while the log scale can improve balance across very different concentration ranges.

4.2 Dissolution Comparison (f_2)

This tool is an IVIVC-flavored wrapper around the shared dissolution-comparison helpers in common.py. It supports FDA-style point selection, f_2 calculation, FDA applicability checks, optional bootstrap/BCa intervals, and dissolution profile plotting.

Implemented equation:

$f_2 = 50 \cdot \log10([1 + (1/n) \Sigma(Rt-Tt)^2]^{-1/2} \cdot 100)$

4.3 In Vitro Weibull Fit

The tool fits single-, double-, and triple-Weibull models to one or more dissolution profiles, compares them by AIC, and optionally saves the best model as InVitroFit. Starting values and bounds can be edited in a table before fitting. The save-model tab also reports approximate confidence and prediction bands around the fitted profile.

Implemented cumulative release equations:

Single Weibull: $F(t)=Fmax\cdot(1-\exp[-(t/MDT1)^{\beta 1}])$

Double Weibull: $F(t)=Fmax\cdot(1-f1\cdot\exp[-(t/MDT1)^{\beta 1}]-(1-f1)\cdot\exp[-(t/MDT2)^{\beta 2}])$

Triple Weibull: same structure with fractions f1, f2, and f3=(1−f1−f2), with a third MDT/β pair.

Model selection is not the optimizer target. The optimizer minimizes residual sum of squares under the chosen scale/weighting scheme; AIC and BIC are then computed afterward to select the preferred Weibull model.

4.4 Deconvolution through convolution

This tool directly fits a release model to PK data by embedding the analytical Weibull input rate inside a compartment ODE system. In practice, it reverses the usual process: instead of starting from a deconvoluted release profile, it searches for Weibull parameters that, after ODE simulation, reproduce the observed PK profile. The best model can be stored as InVivoFit.

Key implementation details:

Component	Implementation
Input rate	Analytical derivative of the Weibull cumulative release, not a finite-difference approximation.
State equations	1-, 2-, or 3-compartment microconstant model written in amounts A1, A2, A3.
Cp conversion	Central amount divided by V and converted from mg/L to the selected Cp unit.
Deconvolution reference	Wagner–Nelson for 1 compartment; Loo–Riegelman-style peripheral reconstruction for 2–3 compartments.
Comparison metric	Weighted/log least-squares RSS, then AIC/BIC for model ranking.

Analytical rate formula used in the ODE: $kab(t) = (\beta/MDT)\cdot(t/MDT)^{(\beta-1)}\cdot\exp(-(t/MDT)^{\beta})$*, multiplied by the appropriate Weibull fractions and by Fmax/100.*

For a 2-compartment system, the code uses: dA1/dt = Input − (k10+k12)A1 + k21A2 and dA2/dt = k12A1 − k21A2. The 3-compartment branch extends this with k13 and k31 and the A3 state.

4.5 IVIVC

The IVIVC tool applies time scaling to a saved in vitro Weibull model and validates it either against a saved InVivoFit release profile (preferred, when available) or directly against the mean PK profile. This makes the IVIVC page usable in two modes: release-space fitting or PK-target fitting.

Core transformation and mapping:

$t'' = B1 + B2\cdot t^{B3}$

The transformed in vitro release is evaluated at t'' using the saved in vitro Weibull parameters. Depending on settings, the model may also estimate A1 and A2 to map the in vitro unreleased fraction to the in vivo unreleased fraction, and it may estimate BIO when the fit target is the PK profile rather than a saved release profile.

The tool also builds additional validation graphics: PK overlay, saved/deconvoluted release overlay, Fabs vs scaled Fdissolved, and scaled-time comparison. Prediction error tables compare PK quantities from the observed mean profile versus the fitted profile (for example Cmax, Tmax, AUCt, AUCinf, λz, and AUC extrapolated %).

Core equations implemented in the app

Dissolution similarity (f_2)

$$f_2 = 50 \log_{10}\left(\left[1 + \frac{1}{n}\sum_{t=1}^{n}(R_t - T_t)^2\right]^{-1/2} \times 100\right)$$

Single Weibull

$$F(t) = F_{max}\left(1 - e^{-(t/MDT_1)^{\beta_1}}\right)$$

Double Weibull

$$F(t) = F_{max}\left(1 - f_1 e^{-(t/MDT_1)^{\beta_1}} - (1 - f_1)e^{-(t/MDT_2)^{\beta_2}}\right)$$

Analytical Weibull input rate

$$k_{ab}(t) = \frac{\beta}{MDT}(t/MDT)^{\beta - 1}e^{-(t/MDT)^{\beta}}$$

1-compartment PK ODE

$$\frac{dA_1}{dt} = Dose \cdot F \cdot k_{ab}(t) - k_{10}A_1$$

2-compartment PK ODE

$$\frac{dA_1}{dt} = Input - (k_{10} + k_{12})A_1 + k_{21}A_2, \quad \frac{dA_2}{dt} = k_{12}A_1 - k_{21}A_2$$

3-compartment PK ODE

$$\frac{dA_1}{dt} = Input - (k_{10} + k_{12} + k_{13})A_1 + k_{21}A_2 + k_{31}A_3$$

IVIVC time scaling

$$t'' = B_1 + B_2 t^{B_3}$$

Wagner–Nelson

$$F_{abs}(t) = \frac{C_t + k_{el}AUC_0^t}{k_{el}AUC_{inf}}$$

Weighted residuals

$$r_i = \sqrt{w_i}\,[\hat{y}_i - y_i], \quad w_i \propto y^{-p} \text{ or } \hat{y}^{-p}$$

Figure 4. Selected equations explicitly implemented or approximated in the current code base.

5. DoE Studio

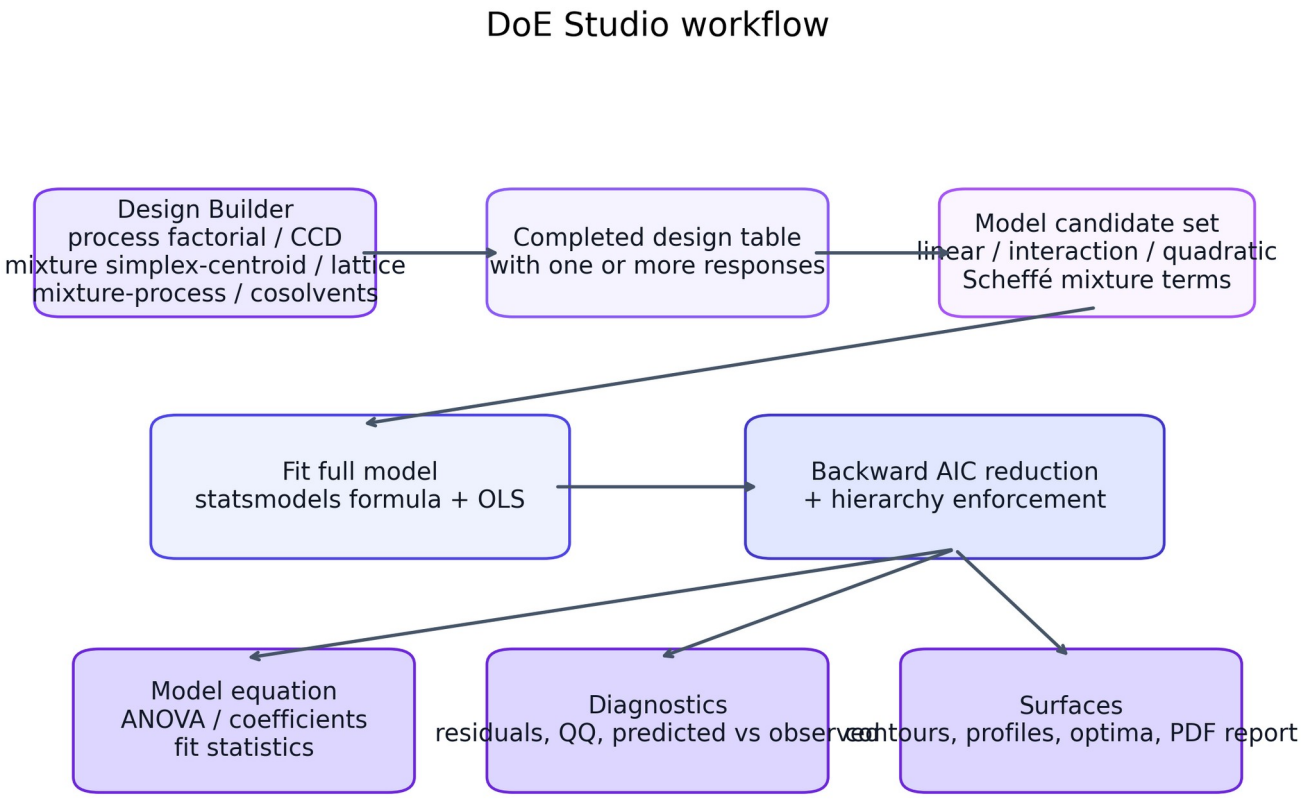

Figure 5. DoE Studio from design generation to hierarchy-aware response-surface analysis and reporting.

DoE Studio is split into two large tabs: Design Builder and Analyze Responses. It supports classical process-factor designs, constrained mixture designs, mixture-process studies, and co-solvent screening layouts that are especially relevant for formulation work.

5.1 Design Builder

The design-family selector currently includes Process factorial, Process CCD, Mixture simplex-centroid, Mixture simplex-lattice, Mixture-Process, Co-Solvents Evaluation, and Co-Solvents Evaluation – Process. The builder can randomize within block, replicate runs, seed the randomization, and emit a CSV design table.

5.2 Response Analysis

The analysis tab routes the completed dataset into one of three families: process, mixture, or mixture-process. The software constructs candidate terms, fits a full model with statsmodels formula syntax, applies backward AIC reduction, and then enforces hierarchy so retained higher-order effects keep the corresponding parent terms.

Model families represented in the code include:

Family	Typical model structure	Notes
Process	linear / interaction / quadratic terms	Suitable for independent numeric process factors.
Mixture	Scheffé-style linear / quadratic / special cubic	Suitable when component fractions sum to a constant.
Mixture-process	Mixture terms + process terms + cross terms	Combines constrained blend effects with independent process factors.

Outputs include coefficient tables, fit statistics, ANOVA, lack-of-fit evaluation where applicable, model equations, residual diagnostics, predicted-vs-observed plots, contour maps, 3D surfaces, mixture profiles, process profiles, overlay contour plots, and optimum point searches.

6. Reporting, export, and table-copy workflow

Across the application, the main reporting pipeline is centralized. report_table() renders a styled HTML table in the app and adds a copy-table control. export_results() packages the selected tables and figures into an Excel workbook and a reportlab-generated PDF report. Figures are converted to PNG bytes through fig_to_png_bytes() before being embedded into the PDF report or downloadable bundle.

The practical consequence is that every analytical page can serve as both an interactive screen and a report generator. Tables are not just visual summaries: they are first-class export objects, and the app keeps a clear table_map / figure_map pattern throughout the modules.

Feature	Implementation command(s)	Comment
Copy table to clipboard	report_table() + browser-side clipboard copy helper	Designed for direct transfer into Word or similar report editors.
Excel export	make_excel_bytes(), st.download_button	Tables are written to a workbook with one sheet per table.
PDF report export	make_pdf_report(), export_results()	Uses reportlab tables, paragraphs, images, and page breaks.
Figure export	fig_to_png_bytes()	Converts matplotlib figures to in-memory PNGs for reports.

7. Python packages, key commands, and runtime APIs

Package	Observed use in the project
streamlit	Multipage navigation, widgets, sidebar, session state, tables, plots, downloads.
pandas	Tabular storage, numeric coercion, grouping, summaries, export staging.
numpy	Vectorized math, array transforms, weighting, interpolation, grid generation.
matplotlib	All application figures, diagnostic plots, surfaces, profiles, and equation/diagram assets.
scipy	Statistical tests, least_squares optimizer, solve_ivp ODE solver, distributions, linregress.
statsmodels	ANOVA, formula-based OLS models, Anderson–Darling normality, Tukey HSD.
scikit-learn	PCA decomposition in the PCA Analysis tool.
openpyxl	Excel workbook generation for exported results.
reportlab	PDF report assembly with styled tables, paragraphs, and figures.

Important commands and signatures found in the code base:

Example command / call	Role
streamlit run app.py	Launches the application locally.
st.navigation([...])	Builds the page navigation tree in app.py.
least_squares(..., method="trf")	Performs bounded nonlinear least-squares fitting.
solve_ivp(..., method="LSODA")	Integrates compartment ODEs for PK and IVIVC workflows.
smf.ols(formula, data).fit()	Fits formula-based OLS models in the GLM / DoE tools.
anova_lm(model, typ=2)	Creates ANOVA tables for fitted models.
pairwise_tukeyhsd(...)	Computes Tukey HSD multiple-comparison results.
PCA(n_components=2).fit_transform(Z)	Computes PCA scores for standardized data.

8. Persistence, saved models, and session behavior

The current IVIVC module exposes three important session objects: InVitroFit, InVivoFit, and IVIVCModel. These are stored in st.session_state and are surfaced in the sidebar as saved-profile indicators. InVitroFit

stores the best Weibull model from the dissolution fit page. InVivoFit stores the best release model and associated PK summary from deconvolution. IVIVCModel stores the final time-scaling fit result, including links back to the in vitro and in vivo sources used.

Because the code relies on Streamlit session state, these objects are available while the session is alive and can be reused when moving between pages. The IVIVC page is designed specifically to consume InVitroFit and, when available, InVivoFit.

9. Appendix A — Module inventory

The following tables summarize the main imported libraries and representative function groups in each module. The purpose is not to mirror every source line, but to give maintainers and advanced users a fast map of where each class of computation lives.

Module	Top-level imports	Top-level functions	Lines reviewed
modules/common.py	23	53	1134
modules/stats_suite.py	3	19	1818
modules/ivivc_suite.py	6	121	3434
modules/doe_studio.py	5	80	2609

Representative function groups

Module	Function group	Examples
common.py	Parsing and formatting	parse_pasted_table, parse_xy, report_table, export_results, apply_ax_style
stats_suite.py	Statistical workflows	_one_sample_summary, regression_anova_and_coefficients_local, _anova_multi_groups, _tukey_pairwise_figure, render
ivivc_suite.py	Fitting / ODE / saved models	fit_weibull_suite, fit_pk_deconvolution_suite, fit_ivivc_tool, _pk_ode_rhs_analytical, save_invivofit_to_session
doe_studio.py	Design generation and RSM	_build_factorial_design, _build_ccd_design, _build_mixture_design, _fit_model_general, _backward_aic_stepwise_general

10. Appendix B — Quick operator checklist

• Launch locally with: streamlit run app.py

• Use the sidebar navigation to select Welcome, Stats Suite, IVIVC Suite, or DoE Studio.

• Paste tables with headers whenever possible; the parser assumes the first row contains column names in most workflows.

• For Weibull / deconvolution / IVIVC tasks, confirm the selected time unit and Cp unit before fitting.

• Use starting-value editors and Fix checkboxes when you want to constrain a fit.

• Save InVitroFit before attempting IVIVC. Save InVivoFit when release-space IVIVC comparison is desired.

• Check the model-comparison tables (AIC/BIC) after fitting; the best visual fit is not always the best-ranked model.

• Use the copy-table controls for direct transfer into Word, and use the export buttons when you need Excel or a PDF report.

End of manual.